MTech – Data Engineering

Machine Learning with Big Data

Assignment 3

Recommender Systems

Submited By:

Arpita Kundu

Roll No: M25DE1004

INDIAN INSTITUTE OF TECHNOLOGY JODHPUR

NH 62 Nagaur Road, Karwar, Jodhpur, Rajasthan

Part 1: Content-Based Filtering

Task 1: Implementing TF-IDF Based Recommendation

```python
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```python
movies_data = pd.read_csv('/content/drive/MyDrive/ml-latest-small/ml-latest-small/movies.csv')

print(f'Dataset shape  : {movies_data.shape}')
print(f'Columns        : {movies_data.columns.tolist()}')
movies_data.head(10)
```

```
Dataset shape  : (9742, 3)
Columns        : ['movieId', 'title', 'genres']
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy
5	6	Heat (1995)	Action|Crime|Thriller
6	7	Sabrina (1995)	Comedy|Romance
7	8	Tom and Huck (1995)	Adventure|Children
8	9	Sudden Death (1995)	Action
9	10	GoldenEye (1995)	Action|Adventure|Thriller

```python
# Pre-process genres:
#   1. Drop rows with no listed genres
#   2. Replace '|' with ' ' so each genre becomes a separate token

#   e.g. "Action|Sci-Fi|Thriller" -> "Action Sci-Fi Thriller"

movies_data = movies_data[movies_data['genres'] != '(no genres listed)'].copy()
movies_data['genres_clean'] = movies_data['genres'].str.replace('|', ' ', regex=False)
movies_data = movies_data.reset_index(drop=True)

print(f'Movies after filtering : {len(movies_data)}')
print('\nSample genre strings:')
print(movies_data[['title', 'genres_clean']].head(8).to_string(index=False))
```

```
Movies after filtering : 9708

Sample genre strings:
                         title                              genres_clean
              Toy Story (1995) Adventure Animation Children Comedy Fantasy
                Jumanji (1995)                 Adventure Children Fantasy
        Grumpier Old Men (1995)                              Comedy Romance
       Waiting to Exhale (1995)                        Comedy Drama Romance
Father of the Bride Part II (1995)                                   Comedy
                   Heat (1995)                        Action Crime Thriller
                Sabrina (1995)                                Comedy Romance
            Tom and Huck (1995)                           Adventure Children
```

step-2. Compute TF-IDF vectors using TfidfVectorizer from sklearn

```python
# TF-IDF Vectorization

tfidf = TfidfVectorizer(
    analyzer='word',
    ngram_range=(1, 1),
    min_df=1,
    stop_words='english'
)

tfidf_matrix = tfidf.fit_transform(movies_data['genres_clean'])

print(f'TF-IDF matrix shape : {tfidf_matrix.shape}')
print(f'  Rows = number of movies          : {tfidf_matrix.shape[0]}')
print(f'  Cols = number of unique genres   : {tfidf_matrix.shape[1]}')
print(f'\nGenre vocabulary ({tfidf_matrix.shape[1]} tokens):')
print(tfidf.get_feature_names_out().tolist())
```

```
TF-IDF matrix shape : (9708, 21)
  Rows = number of movies          : 9708
  Cols = number of unique genres   : 21

Genre vocabulary (21 tokens):
['action', 'adventure', 'animation', 'children', 'comedy', 'crime', 'documentary', 'drama', 'fantasy', 'fi', 'film', 'horror', 'imax', 'musical', 'mystery', 'noir', 'romance', 'sci', 'thriller', 'war', 'western']
```

step-3. Compute cosine similarity between items (may use sklearn.metrics)

```
# Pairwise cosine similarity across all movies

cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)

cosine_sim
```

```
array([[1.        , 0.8136036 , 0.15259961, ..., 0.        , 0.42114166,
        0.26738778],
       [0.8136036 , 1.        , 0.        , ..., 0.        , 0.        ,
        0.        ],
       [0.15259961, 0.        , 1.        , ..., 0.        , 0.        ,
        0.57070525],
       ...,
       [0.        , 0.        , 0.        , ..., 1.        , 0.        ,
        0.        ],
       [0.42114166, 0.        , 0.        , ..., 0.        , 1.        ,
        0.        ],
       [0.26738778, 0.        , 0.57070525, ..., 0.        , 0.        ,
        1.        ]])
```

```
print(f'Cosine similarity matrix shape  : {cosine_sim.shape}')
print(f'Diagonal max (self-similarity)  : {cosine_sim.diagonal().max():.4f}')  # always 1.0
print(f'Overall value range             : [{cosine_sim.min():.4f}, {cosine_sim.max():.4f}]')



# Reverse-lookup
title_to_idx = pd.Series(movies_data.index, index=movies_data['title']).drop_duplicates()
print(f'\nTitle index mapping built for {len(title_to_idx):,} movies.')
```

```
Cosine similarity matrix shape  : (9708, 9708)
Diagonal max (self-similarity)  : 1.0000
Overall value range             : [0.0000, 1.0000]

Title index mapping built for 9,708 movies.
```

step-4.Implement a Recommendation Function : Create a function that takes a movie title and returns the top-N most similar movies.

```
def get_recommendations(title: str, top_n: int = 5) -> pd.DataFrame:

    if title not in title_to_idx:
        raise ValueError(
            f"Movie '{title}' not found in data server.\n"
            f"Tip: use search_title() to find the exact title string."
        )

    idx = title_to_idx[title]
    sim_scores = list(enumerate(cosine_sim[idx]))

    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
    sim_scores = [(i, score) for i, score in sim_scores if i != idx]

    # Take top-N
    top_scores = sim_scores[:top_n]

    # Build a readable results DataFrame
    results = pd.DataFrame([
        {
            'rank': rank + 1,
            'title': movies_data.iloc[i]['title'],
            'genres': movies_data.iloc[i]['genres'],
            'cosine_similarity': round(score, 4)
        }
        for rank, (i, score) in enumerate(top_scores)
    ])

    return results
```

step-5 : Test your recommender with sample queries : Input a few movie titles and print the top-5 recommended movies along with their cosine similarity scores.

```python
samples = [
    "Dark Knight, The (2008)",
    "Schindler's List (1993)",
    "Matrix, The (1999)",
    "Waiting to Exhale (1995)",
    "Geek Girl (2024)"
]

for movie in samples:
    print('=' * 120)
    try:
        recs = get_recommendations(movie, top_n=5)
        genre_str = movies_data.loc[title_to_idx[movie], 'genres']
        print(f'✅ Query : {movie}')
        print(f'   Genres: {genre_str}')
        print('-' * 80)
        print(recs.to_string(index=False))
    except ValueError as e:
        print(f'❌ {e}')
        keyword = movie.split()[0]
        suggestions = search_title(keyword, max_results=3)
        if len(suggestions):
            print(f'   Did you mean one of these?')
            for s in suggestions:
                print(f'     → {s}')

print('=' * 120)
```

```
========================================================================================================================
✅ Query : Dark Knight, The (2008)
   Genres: Action|Crime|Drama|IMAX
--------------------------------------------------------------------------------
 rank                                        title                          genres  cosine_similarity
    1                          Need for Speed (2014)          Action|Crime|Drama|IMAX             1.0000
    2                           Batman Begins (2005)                Action|Crime|IMAX             0.9642
    3 Fast Five (Fast and the Furious 5, The) (2011) Action|Crime|Drama|Thriller|IMAX             0.9322
    4                                Eagle Eye (2008)       Action|Crime|Thriller|IMAX             0.8944
    5                   Good Day to Die Hard, A (2013)       Action|Crime|Thriller|IMAX             0.8944
========================================================================================================================
✅ Query : Schindler's List (1993)
   Genres: Drama|War
--------------------------------------------------------------------------------
 rank                                       title    genres  cosine_similarity
    1                           Richard III (1995) Drama|War                1.0
    2                       Misérables, Les (1995) Drama|War                1.0
    3        Before the Rain (Pred dozhdot) (1994) Drama|War                1.0
    4                     Walking Dead, The (1995) Drama|War                1.0
    5 Land and Freedom (Tierra y libertad) (1995) Drama|War                1.0
========================================================================================================================
✅ Query : Matrix, The (1999)
   Genres: Action|Sci-Fi|Thriller
--------------------------------------------------------------------------------
 rank                                     title                genres  cosine_similarity
    1 Lawnmower Man 2: Beyond Cyberspace (1996) Action|Sci-Fi|Thriller                1.0
    2                          Screamers (1995) Action|Sci-Fi|Thriller                1.0
    3                    Johnny Mnemonic (1995) Action|Sci-Fi|Thriller                1.0
    4                         Virtuosity (1995) Action|Sci-Fi|Thriller                1.0
    5                            Timecop (1994) Action|Sci-Fi|Thriller                1.0
========================================================================================================================
✅ Query : Waiting to Exhale (1995)
   Genres: Comedy|Drama|Romance
--------------------------------------------------------------------------------
 rank                           title                genres  cosine_similarity
    1    American President, The (1995) Comedy|Drama|Romance                1.0
    2           Mighty Aphrodite (1995) Comedy|Drama|Romance                1.0
```

Task 2: User-Profile-Based Content Recommender

Task 2: User-Profile-Based Content Recommender

```python
# Load ratings.csv and filter to movies present in movies DataFrame

ratings = pd.read_csv('/content/drive/MyDrive/ml-latest-small/ml-latest-small/ratings.csv')

movie_indices = movies_data.set_index('movieId').index
ratings = ratings[ratings['movieId'].isin(movie_indices)].copy()

print(f'Ratings loaded : {len(ratings):,}')
print(f'Unique users   : {ratings["userId"].nunique():,}')
print(f'Unique movies  : {ratings["movieId"].nunique():,}')
print(f'Rating range   : {ratings["rating"].min()} – {ratings["rating"].max()}')
ratings.head()
```

```
Ratings loaded : 100,789
Unique users   : 610
Unique movies  : 9,690
Rating range   : 0.5 – 5.0
```

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

Construct User Profiles :

- Highly-rated movies contribute more to the profile.
- Poorly-rated movies contribute less.

```python
movieId_to_idx = pd.Series(movies_data.index, index=movies_data['movieId'])

# Compute user profile for every user
user_profiles = {}

for user_id in ratings['userId'].unique():
    user_data = ratings[ratings['userId'] == user_id]
    tfidf_vectors = []
    user_ratings  = []

    for _, row in user_data.iterrows():
        movie_idx = movieId_to_idx.get(row['movieId'])
        if movie_idx is not None:
            # Weight the TF-IDF vector by the user's rating
            tfidf_vector = tfidf_matrix[movie_idx].toarray().flatten()
            tfidf_vectors.append(tfidf_vector * row['rating'])
            user_ratings.append(row['rating'])

    if tfidf_vectors:
        # Normalise by total rating sum
        profile = np.sum(tfidf_vectors, axis=0) / np.sum(user_ratings)
        user_profiles[user_id] = profile

print(f'User profiles built: {len(user_profiles):,}')

print(f'Profile vector shape (User 1): {user_profiles[1].shape}')
print(f'Top genre weights for User 1:')
top_genre_idx = np.argsort(user_profiles[1])[::-1][:5]
for i in top_genre_idx:
    print(f'  {tfidf.get_feature_names_out()[i]:20s} : {user_profiles[1][i]:.4f}')
```

```
User profiles built: 610
Profile vector shape (User 1): (21,)
Top genre weights for User 1:
  adventure           : 0.1951
  action              : 0.1895
  comedy              : 0.1725
  drama               : 0.1234
  crime               : 0.1228
```

```python
def recommend_for_user(user_id: int, top_n: int = 5) -> list:

    if user_id not in user_profiles:
        return f'No profile found for user {user_id}'

    user_vector = user_profiles[user_id]

    movie_vectors = tfidf_matrix.toarray()
    sims = cosine_similarity([user_vector], movie_vectors).flatten()

    # Movies already rated by the user (exclude from recommendations)
    rated_movie_ids = ratings[ratings['userId'] == user_id]['movieId'].tolist()

    # Candidate indices: movies NOT yet rated
    candidate_indices = [
        i for i, mid in enumerate(movies_data['movieId'])
        if mid not in rated_movie_ids
    ]

    # Sort candidates by similarity score (descending)
    top_indices = sorted(candidate_indices, key=lambda i: sims[i], reverse=True)[:top_n]

    # Return (movieId, title, similarity)
    recommendations = [
        (movies_data.iloc[i]['movieId'], movies_data.iloc[i]['title'], round(sims[i], 3))
        for i in top_indices
    ]
    return recommendations
```

```python
# Run recommendations for User 1 and User 2

for uid in [1, 2]:
    print(f'\nTop recommendations for User {uid}:')
    recs = recommend_for_user(uid, top_n=5)
    for _, title, score in recs:
        print(f'  - {title} (Similarity: {score})')
```

```
Top recommendations for User 1:
  - Dragonheart 2: A New Beginning (2000) (Similarity: 0.791)
  - Hunting Party, The (2007) (Similarity: 0.774)
  - Flashback (1990) (Similarity: 0.773)
  - The Great Train Robbery (1978) (Similarity: 0.773)
  - Charlie's Angels: Full Throttle (2003) (Similarity: 0.76)

Top recommendations for User 2:
  - Money Train (1995) (Similarity: 0.875)
  - Bad Boys (1995) (Similarity: 0.875)
  - Metro (1997) (Similarity: 0.875)
  - Another 48 Hrs. (1990) (Similarity: 0.875)
  - Warabi (2001) (Similarity: 0.875)
```

```python
# Inspect user 1 in detail: top-10 recommendations + liked movies

user_id = 1

recs = recommend_for_user(user_id, top_n=10)
print(f'User ID: {user_id}')
print(f'Recommendations (movieId, title, similarity):')
for r in recs:
    print(f'  {r}')

# Show movies the user liked (rated >= 4)
liked = ratings[(ratings['userId'] == user_id) & (ratings['rating'] >= 4)]
liked_ids = liked['movieId'].tolist()
print(f'\nLiked movie IDs by user {user_id} (rating ≥ 4):')
print(liked_ids)
```

```
User ID: 1
Recommendations (movieId, title, similarity):
  (np.int64(117646), 'Dragonheart I: A New Beginning (2000)', np.float64(0.791))
  (np.int64(55116), 'Hunting Party, The (2007)', np.float64(0.774))
  (np.int64(5657), 'Flashback (1990)', np.float64(0.773))
  (np.int64(6590), 'The Great Train Robbery (1978)', np.float64(0.773))
  (np.int64(6503), "Charlie's Angels: Full Throttle (2003)", np.float64(0.76))
  (np.int64(89648), 'After the Sunset (2004)', np.float64(0.761))
  (np.int64(26114), 'Diamond Arm, The (Brilliantovaya ruka) (1968)', np.float64(0.76))
  (np.int64(88219), 'Machete (2010)', np.float64(0.76))
  (np.int64(164226), 'Maximum Ride (2016)', np.float64(0.76))
  (np.int64(4956), 'Stunt Man, The (1980)', np.float64(0.754))

Liked movie IDs by user 1 (rating ≥ 4):
[1, 3, 6, 47, 50, 101, 110, 151, 157, 163, 216, 231, 235, 260, 333, 349, 356, 362, 367, 441, 457, 480, 527, 543, 552, 553, 590, 592, 593, 596, 608, 661, 733, 804, 919, 923, 940, 943, 954, 1023, 1024, 1025, 1029, 1031, 1032, 1042, 1049, 1060, 1073, 1080, 1089, 1090, 1092, 1097, 1127, 1136, 1196, 1197, 1198,
```

```python
def precision_recall_at_k(user_id: int, k: int = 5, threshold: float = 4.0):

    user_data = ratings[ratings['userId'] == user_id].copy()
    liked     = user_data[user_data['rating'] >= threshold]


    if len(liked) < 2:
        return None, None


    test_movie_ids = set(liked['movieId'].tolist())

    # Train set: non-liked ratings only
    train_data = user_data[~user_data['movieId'].isin(test_movie_ids)]

    # Build a temporary profile from training data
    tfidf_vectors  = []
    user_ratings_list = []
    for _, row in train_data.iterrows():
        movie_idx = movieId_to_idx.get(row['movieId'])
        if movie_idx is not None:
            vec = tfidf_matrix[movie_idx].toarray().flatten()
            tfidf_vectors.append(vec * row['rating'])
            user_ratings_list.append(row['rating'])

    if not tfidf_vectors:
        return None, None

    profile = np.sum(tfidf_vectors, axis=0) / np.sum(user_ratings_list)

    # Compute similarities
    movie_vectors = tfidf_matrix.toarray()
    sims = cosine_similarity([profile], movie_vectors).flatten()

    # Candidates: movies NOT in training set (held-out liked movies are now candidates)
    train_ids = set(train_data['movieId'].tolist())
    candidate_indices = [
        i for i, mid in enumerate(movies_data['movieId'])
        if mid not in train_ids
    ]

    top_indices      = sorted(candidate_indices, key=lambda i: sims[i], reverse=True)[:k]
    recommended_ids  = set(movies_data.iloc[i]['movieId'] for i in top_indices)

    hits      = len(recommended_ids & test_movie_ids)
    precision = hits / k
    recall    = hits / len(test_movie_ids)
    return precision, recall
```

Evaluating Personalization

```
# Precision & Recall across varying K values (K = 1, 5, 10, 20)

print(f'{"K":>4}  {"Precision@K":>12}  {"Recall@K":>10}')
print('-' * 32)

for K in [1, 5, 10, 20]:
    ps, rs = [], []
    for uid in ratings['userId'].unique():
        p, r = precision_recall_at_k(uid, k=K, threshold=4.0)
        if p is not None:
            ps.append(p); rs.append(r)
    print(f'{K:>4}  {np.mean(ps):>12.4f}  {np.mean(rs):>10.4f}')
```

```
   K   Precision@K    Recall@K
--------------------------------
   1       0.0281      0.0005
   5       0.0388      0.0028
  10       0.0286      0.0054
  20       0.0210      0.0106
```

```
user_id = 1
recs = recommend_for_user(user_id, top_n=10)
print("User ID:", user_id)
print("Recommendations (movieId,title, sim):", recs)
```

```
User ID: 1
Recommendations (movieId,title, sim): [(np.int64(117646), 'Dragonheart 2: A New Beginning (2000)', np.float64(0.791)), (np.int64(55116), 'Hunting Party, The (2007)', np.float64(0.774)), (np.int64(5657), 'Flashback (1990)', np.float64(0.773)), (np.int64(6990), 'The Great Train Robbery (1978)', np.float64(0.
```

```
liked = ratings[(ratings['userId'] == user_id) & (ratings['rating'] >= 4)]
print("Liked movieIds by user:",user_id, liked['movieId'].tolist())
```

```
Liked movieIds by user: 1 [1, 3, 6, 47, 50, 101, 110, 153, 157, 163, 216, 231, 235, 260, 333, 349, 356, 362, 367, 441, 457, 480, 527, 543, 552, 553, 590, 592, 593, 596, 608, 661, 733, 804, 919, 923, 940, 943, 954, 1023, 1024, 1025, 1029, 1031, 1032, 1042, 1049, 1060, 1073, 1080, 1089, 1090, 1092, 1097, 112
```

Part 2: Collaborative Filtering

Task 3: User-Based Collaborative Filtering

Step-1. Populate a user-movie rating matrix from the dataset

```
col_fil_ratings = pd.read_csv('/content/drive/MyDrive/ml-latest-small/ml-latest-small/ratings.csv')

movies_data  = movies_data[movies_data['genres'] != '(no genres listed)'].copy().reset_index(drop=True)
movie_indices = movies_data.set_index('movieId').index
col_fil_ratings = col_fil_ratings[col_fil_ratings['movieId'].isin(movie_indices)].copy()

print(f'Movies  : {len(movies_data):,}')
print(f'Ratings : {len(col_fil_ratings):,}')
print(f'Users   : {col_fil_ratings["userId"].nunique():,}')
col_fil_ratings.head()
```

```
Movies  : 9,708
Ratings : 100,789
Users   : 610
```

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

Step-2. Build the User-Movie Rating Matrix

```
top_users  = col_fil_ratings['userId'].value_counts().head(300).index
top_movies = col_fil_ratings['movieId'].value_counts().head(500).index
r_sub = col_fil_ratings[
    col_fil_ratings['userId'].isin(top_users) &
    col_fil_ratings['movieId'].isin(top_movies)
]

# Pivot to user × movie matrix
user_movie_matrix = r_sub.pivot_table(index='userId', columns='movieId', values='rating')

sparsity = 100 * user_movie_matrix.isna().sum().sum() / user_movie_matrix.size
print(f'Matrix shape : {user_movie_matrix.shape}  ({user_movie_matrix.shape[0]} users × {user_movie_matrix.shape[1]} movies)')
print(f'Sparsity     : {sparsity:.1f}%')
print(f'Ratings used : {r_sub.shape[0]:,}')
user_movie_matrix.iloc[:5, :8]
```

```
Matrix shape : (300, 500)  (300 users × 500 movies)
Sparsity     : 75.9%
Ratings used : 36,105
```

movieId	1	2	3	5	6	7	10	11
userId								
1	4.0	NaN	4.0	NaN	4.0	NaN	NaN	NaN
4	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
6	NaN	4.0	5.0	5.0	4.0	4.0	3.0	4.0
7	4.5	NaN	NaN	NaN	NaN	NaN	NaN	NaN
10	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

Compute User-User Similarity (Pearson correlation)

```python
# Pearson similarity via mean-centred cosine similarity
#   1. Subtract each user's mean rating  (removes rating bias)
#   2. Fill NaN with 0  (unrated = neutral deviation)
#   3. Compute cosine similarity on the centred matrix

user_means      = user_movie_matrix.mean(axis=1)
matrix_centered = user_movie_matrix.subtract(user_means, axis=0).fillna(0)

pearson_sim = cosine_similarity(matrix_centered)
pearson_df  = pd.DataFrame(
    pearson_sim,
    index=user_movie_matrix.index,
    columns=user_movie_matrix.index
)

print(f'User-user similarity matrix : {pearson_df.shape}')
print('\nTop-5 most similar users to User 1:')
print(
    pearson_df.loc[1]
    .drop(1)
    .sort_values(ascending=False)
    .head(5)
    .to_frame('Pearson Similarity')
)
```

```
User-user similarity matrix : (300, 300)

Top-5 most similar users to User 1:
        Pearson Similarity
userId
19                0.212905
477               0.205113
469               0.187523
57                0.182038
418               0.161322
```

step-3. Predict ratings using weighted average of neighbour deviations

```python
def predict_rating_user_based(user_id: int, movie_id: int, K: int = 20):
    if movie_id not in user_movie_matrix.columns:
        return None
    if user_id not in pearson_df.index:
        return None

    # Users who have rated this movie (excluding the target user)
    rated_users = user_movie_matrix[movie_id].dropna().index
    rated_users = rated_users.drop(user_id, errors='ignore')
    if len(rated_users) == 0:
        return None

    # K most similar users among those who rated the movie
    sims = pearson_df.loc[user_id, rated_users].sort_values(ascending=False).head(K)
    if sims.abs().sum() == 0:
        return None

    # Weighted average of neighbour deviations from their own mean
    nb_ratings   = user_movie_matrix.loc[sims.index, movie_id]
    nb_means     = user_means.loc[sims.index]
    weighted_dev = (sims * (nb_ratings - nb_means)).sum()
    prediction   = user_means[user_id] + weighted_dev / sims.abs().sum()
    return round(float(prediction), 3)


print('predict_rating_user_based() defined.\n')
print('Sample predictions for User 1:')
for mid in user_movie_matrix.columns[:5]:
    pred       = predict_rating_user_based(1, mid, K=20)
    actual     = user_movie_matrix.loc[1, mid] if 1 in user_movie_matrix.index else None
    actual_str = f'{actual:.1f}' if pd.notna(actual) else 'not rated'
    title      = movies_data[movies_data['movieId'] == mid]['title'].values
    title      = title[0] if len(title) else str(mid)
    print(f'  {title[:45]:<45}  actual={actual_str}  predicted={pred}')
```

```
predict_rating_user_based() defined.

Sample predictions for User 1:
  Toy Story (1995)                            actual=4.0  predicted=4.697
  Jumanji (1995)                              actual=not rated  predicted=3.945
  Grumpier Old Men (1995)                     actual=4.0  predicted=4.002
  Father of the Bride Part II (1995)          actual=not rated  predicted=3.48
  Heat (1995)                                 actual=4.0  predicted=4.372
```

step-4.Recommendation function based on predicted ratings

```python
def recommend_movies(user_id, n=10, k=10):
    """Return the top-n movie IDs predicted for user_id using K neighbours."""
    user_rated       = user_movie_matrix.loc[user_id].dropna().index
    movies_to_predict = user_movie_matrix.columns.difference(user_rated)
    predictions      = {
        movie: predict_rating_user_based(user_id, movie, k)
        for movie in movies_to_predict
    }
    # Remove movies where prediction failed
    predictions      = {m: p for m, p in predictions.items() if p is not None}
    sorted_predictions = sorted(predictions.items(), key=lambda x: x[1], reverse=True)
    return [movie_id for movie_id, _ in sorted_predictions[:n]]


# Demo: top-10 recommendations for User 1
recs = recommend_movies(1, n=10, k=20)
print('Top-10 recommendations for User 1:')
for i, mid in enumerate(recs, 1):
    title = movies_data[movies_data['movieId'] == mid]['title'].values
    title = title[0] if len(title) else str(mid)
    pred  = predict_rating_user_based(1, mid, K=20)
    print(f'  {i:2}. {title[:50]:<50}  predicted={pred}')
```

```
Top-10 recommendations for User 1:
   1. Godfather, The (1972)                          predicted=5.368
   2. Shawshank Redemption, The (1994)               predicted=5.342
   3. Godfather: Part II, The (1974)                 predicted=5.321
   4. Dr. Strangelove or: How I Learned to Stop Worrying  predicted=5.312
   5. Casablanca (1942)                              predicted=5.192
   6. Brazil (1985)                                  predicted=5.161
   7. Departed, The (2006)                           predicted=5.146
   8. 12 Angry Men (1957)                            predicted=5.138
   9. Seven Samurai (Shichinin no samurai) (1954)    predicted=5.118
  10. 2001: A Space Odyssey (1968)                   predicted=5.115
```

step-5. Evaluate the system with RMSE, Precision@k and Recall@k

```python
from sklearn.metrics import mean_squared_error

def evaluate_rmse(test_data, k=10):
    predictions = []
    truths      = []
    for _, row in test_data.iterrows():
        pred = predict_rating_user_based(row['userId'], row['movieId'], k)
        if not np.isnan(pred if pred is not None else float('nan')):
            predictions.append(pred)
            truths.append(row['rating'])
    return np.sqrt(mean_squared_error(truths, predictions))
```

step-6. Test with different values of K (number of similar users) to see how recommendation quality changes.

```python
from sklearn.model_selection import train_test_split

train_data, test_data = train_test_split(r_sub, test_size=0.2, random_state=42)

# Rebuild the user-movie matrix from training data only
user_movie_matrix = train_data.pivot_table(index='userId', columns='movieId', values='rating')
user_means        = user_movie_matrix.mean(axis=1)
matrix_centered   = user_movie_matrix.subtract(user_means, axis=0).fillna(0)
pearson_sim       = cosine_similarity(matrix_centered)
pearson_df        = pd.DataFrame(
    pearson_sim,
    index=user_movie_matrix.index,
    columns=user_movie_matrix.index
)

# Restrict test_data to users/movies seen during training
test_data = test_data[
    test_data['userId'].isin(user_movie_matrix.index) &
    test_data['movieId'].isin(user_movie_matrix.columns)
]

rmse_score = evaluate_rmse(test_data, k=10)
print(f'RMSE for K=10: {rmse_score:.4f}')
```

```
RMSE for K=10: 0.8238
```

```python
def precision_recall_at_k(test_data, k=10, n_recs=10, threshold=4.0):
    user_grp = test_data.groupby('userId')
    prec     = []
    recalls  = []
    for user_id, group in user_grp:
        act_liked = set(group[group['rating'] >= threshold]['movieId'])
        if len(act_liked) == 0:
            continue
        rcmd      = recommend_movies(user_id, n=n_recs, k=k)
        rcmd      = set(rcmd)
        tp        = rcmd.intersection(act_liked)
        precision = len(tp) / len(rcmd) if rcmd else 0
        recall    = len(tp) / len(act_liked)
        prec.append(precision)
        recalls.append(recall)
    avg_precision = np.mean(prec)
    avg_recall    = np.mean(recalls)
    return avg_precision, avg_recall


precision, recall = precision_recall_at_k(test_data, k=10, n_recs=10)
print(f'Precision@10: {precision:.4f}')
print(f'Recall@10:    {recall:.4f}')
```

```
Precision@10: 0.1275
Recall@10:    0.0929
```

```python
# Test with different K values
print('K    RMSE       Precision@10  Recall@10')
print('-' * 45)
for k in [5, 10, 20, 40]:
    rmse      = evaluate_rmse(test_data, k=k)
    prec, rec = precision_recall_at_k(test_data, k=k, n_recs=10)
    print(f'{k:<4} {rmse:.4f}     {prec:.4f}        {rec:.4f}')
```

```
K    RMSE       Precision@10  Recall@10
---------------------------------------------
5    0.8493     0.1037        0.0779
10   0.8238     0.1275        0.0929
20   0.8134     0.1352        0.0997
40   0.8122     0.1453        0.1130
```

Task 4: Item-Based Collaborative Filtering

step-1. Compute item-item similarity using Pearson correlation or cosine similarity

```python
# Build a full user-movie matrix (all ratings)
# Rows = userId, Columns = movieId
full_matrix = ratings.pivot_table(
    index='userId', columns='movieId', values='rating'
)

# Item-item Pearson correlation
# corr() operates column-wise, so each column (movie) is correlated with every other
# min_periods=10 ensures at least 10 co-rated users before trusting the correlation
item_similarity = full_matrix.corr(method='pearson', min_periods=10)

print(f'Item-item similarity matrix shape : {item_similarity.shape}')
print('\nTop-5 movies most similar to movieId=1 (Toy Story):')
toy_story_sims = (
    item_similarity[1]
    .drop(1)
    .dropna()
    .sort_values(ascending=False)
    .head(5)
)
for mid, sim in toy_story_sims.items():
    title = movies_data[movies_data['movieId'] == mid]['title'].values
    title = title[0] if len(title) else str(mid)
    print(f'  {title[:50]:<50}  similarity={sim:.4f}')
```

```
Item-item similarity matrix shape : (9690, 9690)

Top-5 movies most similar to movieId=1 (Toy Story):
  22 Jump Street (2014)                              similarity=0.9133
  Untitled Spider-Man Reboot (2017)                  similarity=0.8985
  Darjeeling Limited, The (2007)                     similarity=0.8688
  Mighty Wind, A (2003)                              similarity=0.8576
  Let the Right One In (Låt den rätte komma in) (200  similarity=0.8328
```

step-2. Predict ratings for a given user based on similar items

- Identify items the user has already rated.
- Find similar items and use their ratings to predict new ratings
- Compute the weighted average of sillimar item ratings, where weights are similarity scores.
- Hint: Higher similarity = higher weight in rating prediction.

```python
def predict_item_rating(user_id, movie_id, user_movie_mat, sim_matrix, k=10):
    """
    Predict the rating user `user_id` would give to `movie_id`
    using item-based CF with K nearest similar items.
    """
    # movie must exist in our matrix
    if movie_id not in user_movie_mat.columns:
        return np.nan
    # user must exist
    if user_id not in user_movie_mat.index:
        return np.nan

    # Movies the user has already rated
    user_rated  = user_movie_mat.loc[user_id].dropna()

    # Similarity scores between target movie and all other movies
    sim_scores  = sim_matrix[movie_id].dropna()

    # Keep only movies that both have a similarity score AND were rated by the user
    common_movies = set(user_rated.index).intersection(sim_scores.index)
    if not common_movies:
        return np.nan

    common_list   = list(common_movies)
    sim_common    = sim_scores.loc[common_list]          # similarity values
    ratings_common = user_rated.loc[common_list]          # user's actual ratings

    # Select top-K most similar items (by absolute similarity)
    top_k         = sim_common.abs().sort_values(ascending=False).head(k)

    # Weighted sum: rating * similarity
    weighted_sum  = (ratings_common.loc[top_k.index] * sim_common.loc[top_k.index]).sum()
    sim_sum       = sim_common.loc[top_k.index].abs().sum()
```


```python
    common_list   = list(common_movies)
    sim_common    = sim_scores.loc[common_list]          # similarity values
    ratings_common = user_rated.loc[common_list]          # user's actual ratings

    # Select top-K most similar items (by absolute similarity)
    top_k         = sim_common.abs().sort_values(ascending=False).head(k)

    # Weighted sum: rating * similarity
    weighted_sum  = (ratings_common.loc[top_k.index] * sim_common.loc[top_k.index]).sum()
    sim_sum       = sim_common.loc[top_k.index].abs().sum()
```


```python
    if sim_sum == 0:
        return np.nan

    return weighted_sum / sim_sum


print('Sample item-based predictions for User 1:')
sample_movies = full_matrix.columns[:5]
for mid in sample_movies:
    pred   = predict_item_rating(1, mid, full_matrix, item_similarity, k=10)
    actual = full_matrix.loc[1, mid] if 1 in full_matrix.index else np.nan
    actual_str = f'{actual:.1f}' if pd.notna(actual) else 'not rated'
    title  = movies_data[movies_data['movieId'] == mid]['title'].values
    title  = title[0] if len(title) else str(mid)
    print(f'  {title[:45]:<45}  actual={actual_str}  predicted={round(pred, 3) if not np.isnan(pred) else "N/A"}')
```

```
Sample item-based predictions for User 1:
  Toy Story (1995)                              actual=4.0  predicted=3.826
  Jumanji (1995)                                actual=not rated  predicted=4.396
  Grumpier Old Men (1995)                       actual=4.0  predicted=4.671
  Waiting to Exhale (1995)                      actual=not rated  predicted=N/A
  Father of the Bride Part II (1995)            actual=not rated  predicted=4.059
```

step-3. Generate top-N movie recommendations for a user

```python
def recommend_item_based(user_id, user_movie_mat, sim_matrix, n=10, k=10):
    """
    Return the top-n movie IDs for `user_id` using item-based CF.
    Only considers movies the user has NOT yet rated.
    """
    rated_movies    = user_movie_mat.loc[user_id].dropna().index
    movies_to_score = user_movie_mat.columns.difference(rated_movies)

    predictions = {
        movie: predict_item_rating(user_id, movie, user_movie_mat, sim_matrix, k)
        for movie in movies_to_score
    }
    # Remove movies where we couldn't produce a prediction
    predictions = {m: p for m, p in predictions.items() if not np.isnan(p)}

    sorted_preds = sorted(predictions.items(), key=lambda x: x[1], reverse=True)
    return [movie_id for movie_id, _ in sorted_preds[:n]]


recs = recommend_item_based(1, full_matrix, item_similarity, n=10, k=10)
print('Top-10 item-based recommendations for User 1:')
for rank, mid in enumerate(recs, 1):
    pred  = predict_item_rating(1, mid, full_matrix, item_similarity, k=10)
    title = movies_data[movies_data['movieId'] == mid]['title'].values
    title = title[0] if len(title) else str(mid)
    print(f'  {rank:2}. {title[:50]:<50}  predicted={round(pred, 3)}')
```

```
Top-10 item-based recommendations for User 1:
   1. In the Heat of the Night (1967)                    predicted=5.0
   2. Fistful of Dollars, A (Per un pugno di dollari) (1 predicted=5.0
   3. John Carter (2012)                                 predicted=5.0
   4. Harriet the Spy (1996)                             predicted=5.0
   5. Cat on a Hot Tin Roof (1958)                       predicted=5.0
   6. Parent Trap, The (1961)                            predicted=5.0
   7. American Tail: Fievel Goes West, An (1991)         predicted=5.0
   8. Urban Legend (1998)                                predicted=5.0
   9. Astronaut's Wife, The (1999)                       predicted=5.0
  10. Mister Roberts (1955)                              predicted=5.0
```

step-4. Evaluate the system and compare the recommendations with user-based CF to analyze differences in output.

```python
def evaluate_item_based_rmse(test_data, train_matrix, sim_matrix, k=10):
    """Compute RMSE of item-based CF on held-out test ratings."""
    preds  = []
    truths = []
    for _, row in test_data.iterrows():
        uid, mid = int(row['userId']), int(row['movieId'])
        pred = predict_item_rating(uid, mid, train_matrix, sim_matrix, k)
        if not np.isnan(pred):
            preds.append(pred)
            truths.append(row['rating'])
    return np.sqrt(mean_squared_error(truths, preds))


# — Train / test split
train_data, test_data = train_test_split(ratings, test_size=0.2, random_state=42)

# Build training matrix and recompute item similarity from training data only
train_matrix    = train_data.pivot_table(
    index='userId', columns='movieId', values='rating'
)
train_item_sim  = train_matrix.corr(method='pearson', min_periods=10)

# Filter test set to users / movies seen in training
test_filtered   = test_data[
    test_data['userId'].isin(train_matrix.index) &
    test_data['movieId'].isin(train_matrix.columns)
]

rmse_item = evaluate_item_based_rmse(test_filtered, train_matrix, train_item_sim, k=10)
print(f'Item-Based CF RMSE (k=10): {rmse_item:.4f}')
```

```
Item-Based CF RMSE (k=10): 1.7446
```

```python
# — User-Based CF for comparison
from sklearn.metrics.pairwise import cosine_similarity as cos_sim

# Build Pearson (mean-centred cosine) user similarity from training data
user_means_train   = train_matrix.mean(axis=1)
centered_train     = train_matrix.subtract(user_means_train, axis=0).fillna(0)
user_sim_arr       = cos_sim(centered_train)
user_sim_df        = pd.DataFrame(
    user_sim_arr,
    index=train_matrix.index,
    columns=train_matrix.index
)

def predict_user_based(user_id, movie_id, mat, sim_df, means, k=10):
    if movie_id not in mat.columns or user_id not in sim_df.index:
        return np.nan
    rated_users = mat[movie_id].dropna().index.difference([user_id])
    if len(rated_users) == 0:
        return np.nan
    sims = sim_df.loc[user_id, rated_users].sort_values(ascending=False).head(k)
    if sims.abs().sum() == 0:
        return np.nan
    nb_ratings   = mat.loc[sims.index, movie_id]
    nb_means     = means.loc[sims.index]
    weighted_dev = (sims * (nb_ratings - nb_means)).sum()
    return means[user_id] + weighted_dev / sims.abs().sum()

def evaluate_user_based_rmse(test_data, mat, sim_df, means, k=10):
    preds, truths = [], []
    for _, row in test_data.iterrows():
        pred = predict_user_based(int(row['userId']), int(row['movieId']), mat, sim_df, means, k)
        if pred is not None and not np.isnan(pred):
            preds.append(pred)
            truths.append(row['rating'])
    return np.sqrt(mean_squared_error(truths, preds))

rmse_user = evaluate_user_based_rmse(test_filtered, train_matrix, user_sim_df, user_means_train, k=10)
print(f'User-Based CF RMSE (k=10): {rmse_user:.4f}')
print(f'Item-Based CF RMSE (k=10): {rmse_item:.4f}')
winner = 'Item-Based' if rmse_item < rmse_user else 'User-Based'
print(f'\n→ {winner} CF has lower RMSE on this dataset.')
```

```
User-Based CF RMSE (k=10): 0.8843
Item-Based CF RMSE (k=10): 1.7446

→ User-Based CF has lower RMSE on this dataset.
```

5. Do you think item based CF may be faster or more memory-efficient than user based CF in the real world? Why?

- In real-world scenarios, Item-Based CF is generally more efficient than User-Based CF for several reasons.
- At recommendation time, the system needs a fast lookup and that making Item-Based CF more scalable and production friendly.

Part 3: Matrix Factorization for Recommender Systems

Task 5: Implementing SVD for Recommendations

Task 5: Implementing SVD for Recommendations

step-1. Prepare the user-item matrix for factorization

```
import pandas as pd
import numpy as np
from sklearn.metrics import mean_squared_error
from sklearn.metrics import precision_score, recall_score
ratings = pd.read_csv('/content/drive/MyDrive/ml-latest-small/ml-latest-small/ratings.csv')
user_movie_matrix = ratings.pivot_table(index='userId', columns='movieId', values='rating')
user_means = user_movie_matrix.mean(axis=1)
normalized_matrix = user_movie_matrix.sub(user_means, axis=0).fillna(0)
```

step-2. Apply Singular Value Decomposition (SVD) using scipy or numpy functions

```
from numpy.linalg import svd
U, sigma, VT = svd(normalized_matrix, full_matrices=False)
k = 20
sigma_k = np.diag(sigma[:k])
U_k = U[:, :k]
VT_k = VT[:k, :]
```

step-3. Predict missing ratings

```
reconstructed = np.dot(np.dot(U_k, sigma_k), VT_k)
pred_ratings = reconstructed + user_means.values[:, np.newaxis]
pred_ratings = np.clip(pred_ratings, 0.5, 5.0)
predicted_df = pd.DataFrame(pred_ratings, index=user_movie_matrix.index, columns=user_movie_matrix.columns)
def recommend_svd(user_id, n=10):
    user_row = predicted_df.loc[user_id]
    rated_alredy = user_movie_matrix.loc[user_id].dropna().index
    rmds = user_row.drop(rated_alredy).sort_values(ascending=False).head(n)
    return rmds.index.tolist()
```

step-4. Evaluate SVD-based recommendations and compare with item-based and user-based CF. Use RMSE, Precision@K, and Recall@K

```
from sklearn.model_selection import train_test_split
train, test = train_test_split(ratings, test_size=0.2, random_state=42)
train_matrix = train.pivot_table(index='userId', columns='movieId', values='rating')
user_means = train_matrix.mean(axis=1)
norm_train = train_matrix.sub(user_means, axis=0).fillna(0)
U, sigma, VT = svd(norm_train, full_matrices=False)
U_k = U[:, :k]
sigma_k = np.diag(sigma[:k])
VT_k = VT[:k, :]
reconstructed = np.dot(np.dot(U_k, sigma_k), VT_k)
pred_ratings = reconstructed + user_means.values[:, np.newaxis]
pred_ratings = np.clip(pred_ratings, 0.5, 5.0)
predicted_df = pd.DataFrame(pred_ratings, index=train_matrix.index, columns=train_matrix.columns)
def evaluate_rmse_svd(test_data):
    predictions = []
    truths = []
    for _, row in test_data.iterrows():
        user = row['userId']
        movie = row['movieId']
        if user in predicted_df.index and movie in predicted_df.columns:
            pred = predicted_df.loc[user, movie]
            predictions.append(pred)
            truths.append(row['rating'])
    return np.sqrt(mean_squared_error(truths, predictions))

rmse_svd = evaluate_rmse_svd(test)
print(f"SVD RMSE: {rmse_svd:.4f}")
```

```
SVD RMSE: 0.9207
```

```
def precision_recall_at_k(predicted_df, test_data, k=10, threshold=4.0):
    precision_list = []
    recall_list = []

    test_user_group = test_data.groupby('userId')

    for user_id, group in test_user_group:
        if user_id not in predicted_df.index:
            continue

        relevant_items = group[group['rating'] >= threshold]['movieId'].tolist()

        user_predictions = predicted_df.loc[user_id]
        already_rated = train_matrix.loc[user_id].dropna().index if user_id in train_matrix.index else []
        top_k_recs = user_predictions.drop(index=already_rated, errors='ignore') \
                                     .sort_values(ascending=False) \
                                     .head(k).index.tolist()

        if not relevant_items:
            continue

        hits = len(set(top_k_recs) & set(relevant_items))

        precision = hits / k
        recall = hits / len(relevant_items)
        precision_list.append(precision)
        recall_list.append(recall)

    avg_precision = np.mean(precision_list)
    avg_recall = np.mean(recall_list)

    return avg_precision, avg_recall

precision, recall = precision_recall_at_k(predicted_df, test, k=10)
print(f"Precision@10: {precision:.4f}")
print(f"Recall@10: {recall:.4f}")
```

```
Precision@10: 0.1344
Recall@10: 0.1146
```

Task 6: Implementing Matrix Factorization with the Surprise Library

step-1. Load the dataset into the surprise framework. Split it into train-test sets

```
ratings_df = pd.read_csv('/content/drive/MyDrive/ml-latest-small/ml-latest-small/ratings.csv')
reader = Reader(rating_scale=(0.5, 5.0))
data = Dataset.load_from_df(ratings_df[['userId', 'movieId', 'rating']], reader)
trainset, testset = train_test_split(data, test_size=0.2, random_state=42)
```

step-2. Train an SVD model using surprise and tune the hyperparameters

```
# train SVD model
model = SVD()
model.fit(trainset)
```

```
<surprise.prediction_algorithms.matrix_factorization.SVD at 0x7eca8e323410>
```

step-3. Evaluate model performance with RMSE, Precision@K, and Recall@K

```
predictions = model.test(testset)
rmse = accuracy.rmse(predictions)
```

```
RMSE: 0.8821
```

step-4. Compare Surprise's SVD implementation vs. the SVD implementation from task 5

```
from collections import defaultdict

def get_top_n(predictions, n=10, threshold=4.0):
    top_n = defaultdict(list)

    for uid, iid, true_r, est, _ in predictions:
        if est >= threshold:
            top_n[uid].append((iid, est))

    for uid, user_ratings in top_n.items():
        user_ratings.sort(key=lambda x: x[1], reverse=True)
        top_n[uid] = user_ratings[:n]

    return top_n

def precision_recall_at_k(predictions, k=10, threshold=4.0):
    top_n = get_top_n(predictions, n=k, threshold=threshold)

    user_relevant = defaultdict(set)
    user_recommended = defaultdict(set)

    for uid, iid, true_r, est, _ in predictions:
        if true_r >= threshold:
            user_relevant[uid].add(iid)
        if est >= threshold:
            user_recommended[uid].add(iid)

    precisions = []
    recalls = []

    for uid in user_recommended:
        if uid in user_relevant:
            relevant = user_relevant[uid]
            recommended = user_recommended[uid]
            hits = len(recommended & relevant)

            precisions.append(hits / k)
            recalls.append(hits / len(relevant) if relevant else 0)

    return np.mean(precisions), np.mean(recalls)

precision, recall = precision_recall_at_k(predictions, k=10, threshold=4.0)
print(f"Precision@10: {precision:.4f}")
print(f"Recall@10: {recall:.4f}")
```

```
Precision@10: 0.6993
Recall@10: 0.4398
```

Part 4: Hybrid Recommendation Model

Task 7: Implementing a Hybrid Recommendation Model

Part 4: Hybrid Recommendation Model

Task 7: Implementing a Hybrid Recommendation Model

step-1. Implement a meta-learning strategy.

```python
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.metrics import mean_squared_error
from math import sqrt

movies = pd.read_csv('/content/drive/MyDrive/ml-latest-small/ml-latest-small/movies.csv')
movies['genres'] = movies['genres'].str.replace('|', ' ', regex=False)
tfidf = TfidfVectorizer()
tfidf_matrix = tfidf.fit_transform(movies['genres'].fillna(''))
cbf_similarity = cosine_similarity(tfidf_matrix)
movie_index = {movieId: idx for idx, movieId in enumerate(movies['movieId'])}
index_movie = {idx: movieId for movieId, idx in movie_index.items()}

ratings_df = pd.read_csv('/content/drive/MyDrive/ml-latest-small/ml-latest-small/ratings.csv')
user_movie_matrix = ratings_df.pivot_table(index='userId', columns='movieId', values='rating')

cbf_preds = {}

for uid in user_movie_matrix.index:
    cbf_preds[uid] = {}
    user_ratings = user_movie_matrix.loc[uid]
    rated_movies = user_ratings.dropna().index
    for target_movie in user_movie_matrix.columns:
        if pd.notna(user_movie_matrix.loc[uid, target_movie]):
            continue
        numer, denom = 0, 0
        for rated_movie in rated_movies:
            # Ensure the rated_movie is present in the movie_index to avoid errors
            if rated_movie in movie_index:
                i = movie_index.get(target_movie)
                j = movie_index.get(rated_movie)
                if i is not None and j is not None:
                    sim = cbf_similarity[i, j]
                    numer += sim * user_ratings[rated_movie]
                    denom += abs(sim)

        cbf_preds[uid][target_movie] = numer / denom if denom != 0 else np.nan
```

```python
from sklearn.metrics.pairwise import pairwise_distances
user_similarity = 1 - pairwise_distances(user_movie_matrix.fillna(0), metric='cosine')
user_mean = user_movie_matrix.mean(axis=1)
ratings_diff = user_movie_matrix.sub(user_mean, axis=0).fillna(0)
cf_preds = {}
for i, uid in enumerate(user_movie_matrix.index):
    cf_preds[uid] = {}
    for mid in user_movie_matrix.columns:
        if pd.notna(user_movie_matrix.loc[uid, mid]):
            continue
        numer = 0
        denom = 0
        for j, other_uid in enumerate(user_movie_matrix.index):
            if i == j:
                continue
            sim = user_similarity[i, j]
            if pd.notna(user_movie_matrix.loc[other_uid, mid]):
                numer += sim * (user_movie_matrix.loc[other_uid, mid] - user_mean[other_uid])
                denom += abs(sim)

        pred = user_mean[uid] + numer / denom if denom != 0 else user_mean[uid]
        cf_preds[uid][mid] = pred
```

```python
import pandas as pd
import numpy as np
def create_meta_features(ratings_df, cbf_preds, cf_preds):
    user_avg = ratings_df.groupby('userId')['rating'].mean().to_dict()
    movie_avg = ratings_df.groupby('movieId')['rating'].mean().to_dict()
    features = []
    targets = []
    for _, row in ratings_df.iterrows():
        uid = row['userId']
        mid = row['movieId']
        rating = row['rating']

        if uid in cbf_preds and mid in cbf_preds[uid] and uid in cf_preds and mid in cf_preds[uid]:
            cbf_score = cbf_preds[uid][mid]
            cf_score = cf_preds[uid][mid]
            movie_score = movie_avg.get(mid, 3.0)
            user_score = user_avg.get(uid, 3.0)

            features.append([cbf_score, cf_score, movie_score, user_score])
            targets.append(rating)
    X = np.array(features)
    y = np.array(targets)
    return X, y
```

step-2. Evaluate the hybrid model using RMSE, Precision@K, and Recall@K. Compare against CF and CBF recommender systems

RMSE

```python
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
X, y = create_meta_features(ratings_df, cbf_preds, cf_preds)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
meta_model = RandomForestRegressor(n_estimators=100, random_state=42)
meta_model.fit(X_train, y_train)
y_pred = meta_model.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print(f"Hybrid Model RMSE: {rmse:.4f}")
```

```
Hybrid Model RMSE: 0.8182
```

Precision@K and Recall@K

```python
from collections import defaultdict
def get_top_k_recommendations(meta_model, ratings_df, cbf_preds, cf_preds, k=10):
    user_items = defaultdict(list)
    movie_avg = ratings_df.groupby('movieId')['rating'].mean().to_dict()
    user_avg = ratings_df.groupby('userId')['rating'].mean().to_dict()

    for uid in cf_preds:
        for mid in cf_preds[uid]:
            if mid in cbf_preds[uid]:
                features = [
                    cbf_preds[uid][mid],
                    cf_preds[uid][mid],
                    movie_avg.get(mid, 3.0),
                    user_avg.get(uid, 3.0)
                ]
                pred = meta_model.predict([features])[0]
                user_items[uid].append((mid, pred))
    top_k = {uid: sorted(items, key=lambda x: -x[1])[:k] for uid, items in user_items.items()}
    return top_k
def precision_recall_at_k(top_k_preds, ratings_df, k=10, threshold=3.5):
    actuals = defaultdict(set)
    for _, row in ratings_df.iterrows():
        if row['rating'] >= threshold:
            actuals[row['userId']].add(row['movieId'])
    precision_list = []
    recall_list = []
    for uid, preds in top_k_preds.items():
        if uid in actuals:
            recommended = {mid for mid, _ in preds}
            relevant = actuals[uid]
            hits = recommended & relevant

            if recommended:
                precision = len(hits) / len(recommended)
                recall = len(hits) / len(relevant) if relevant else 0
                precision_list.append(precision)
                recall_list.append(recall)
    return np.mean(precision_list), np.mean(recall_list)
top_k_preds = get_top_k_recommendations(meta_model, ratings_df, cbf_preds, cf_preds, k=10)
precision, recall = precision_recall_at_k(top_k_preds, ratings_df, k=10)
print(f"Precision@10: {precision:.4f}, Recall@10: {recall:.4f}")
```

```
Precision@10: 0.5144, Recall@10: 0.4145
```

step-3. Analyze performance on cold-start users.

```python
user_counts = ratings_df['userId'].value_counts()
cold_start_users = set(user_counts[user_counts < 5].index)
def evaluate_cold_start(top_k_preds, ratings_df, cold_start_users, k=10):
    cold_preds = {uid: preds for uid, preds in top_k_preds.items() if uid in cold_start_users}
    return precision_recall_at_k(cold_preds, ratings_df, k)
cold_precision, cold_recall = evaluate_cold_start(top_k_preds, ratings_df, cold_start_users)

print(f"Cold-start Precision@10: {cold_precision:.4f}, Recall@10: {cold_recall:.4f}")
```

```
Cold-start Precision@10: 0.5456, Recall@10: 0.4859
```

Part 5: Learning-Based Recommender Systems

Task 8: Content-Based Filtering with a Neural Network

step-1. Load and process the movie lens dataset

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Concatenate
from tensorflow.keras.callbacks import EarlyStopping

movies = pd.read_csv("/content/drive/MyDrive/ml-latest-small/ml-latest-small/movies.csv")
movies['genres'] = movies['genres'].str.replace('|', ' ', regex=False)
ratings = pd.read_csv("/content/drive/MyDrive/ml-latest-small/ml-latest-small/ratings.csv")
movies['year'] = movies['title'].str.extract(r'\((\d{4})\)').fillna(2000).astype(int)
genres_split = movies['genres'].str.get_dummies(sep='|')
movie_features = pd.concat([movies[['movieId', 'year']], genres_split], axis=1)
scaler = MinMaxScaler()
movie_features['year'] = scaler.fit_transform(movie_features[['year']])
movie_avg = ratings.groupby('movieId')['rating'].mean().reset_index().rename(columns={'rating': 'avg_rating'})
movie_features = movie_features.merge(movie_avg, on='movieId', how='left').fillna(0)


user_genre_matrix = ratings.merge(movies[['movieId', 'genres']], on='movieId')

user_genres = user_genre_matrix['genres'].str.get_dummies(sep='|')
user_genre_avg = pd.concat([user_genre_matrix[['userId', 'rating']], user_genres], axis=1)
user_genre_avg = user_genre_avg.groupby('userId').mean().reset_index()
ratings_data = ratings.merge(movie_features, on='movieId', how='left')
ratings_data = ratings_data.merge(user_genre_avg, on='userId', how='left')
```

step-2. Create neural networks for generating user and item embeddings. step-3. Prepare the complete neural network

```
movie_input_size = len(genres_split.columns) + 2  # year + avg_rating + genres
user_input_size = len(genres_split.columns) + 1  # genre avg + userId rating avg
user_input = Input(shape=(user_input_size,), name='user_input')
movie_input = Input(shape=(movie_input_size,), name='movie_input')
user_vector = Dense(64, activation='relu')(user_input)
movie_vector = Dense(64, activation='relu')(movie_input)
merged = Concatenate()([user_vector, movie_vector])
hidden = Dense(32, activation='relu')(merged)
output = Dense(1)(hidden)

model = Model(inputs=[user_input, movie_input], outputs=output)
model.compile(optimizer='adam', loss='mse', metrics=['mae'])
model.summary()
```

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
user_input (InputLayer)	(None, 952)	0	-
movie_input (InputLayer)	(None, 953)	0	-
dense (Dense)	(None, 64)	60,992	user_input[0][0]
dense_1 (Dense)	(None, 64)	61,056	movie_input[0][0]
concatenate (Concatenate)	(None, 128)	0	dense[0][0], dense_1[0][0]
dense_2 (Dense)	(None, 32)	4,128	concatenate[0][0]
dense_3 (Dense)	(None, 1)	33	dense_2[0][0]

Total params: 126,209 (493.00 KB)
Trainable params: 126,209 (493.00 KB)
Non-trainable params: 0 (0.00 B)

step-4. Train the model on user ratings

Training

```
genre_movie_cols = [col for col in ratings_data.columns if col.endswith('_x') and col not in ['rating_x', 'year']]
genre_user_cols = [col for col in ratings_data.columns if col.endswith('_y')]
target_ratings = ratings_data['rating_x'].values
user_features = ratings_data[genre_user_cols].values
movie_features_input = ratings_data[['year', 'avg_rating'] + genre_movie_cols].values

X_user_train, X_user_test, X_movie_train, X_movie_test, y_train, y_test = train_test_split(
    user_features, movie_features_input, target_ratings, test_size=0.2, random_state=42
)

early_stopping = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)

model.fit(
    [X_user_train, X_movie_train],
    y_train,
    validation_split=0.1,
    epochs=20,
    batch_size=128,
    callbacks=[early_stopping]
)
```

```
Epoch 1/20
568/568 ━━━━━━━━━━━━━━━━━━━━ 6s 6ms/step - loss: 1.1961 - mae: 0.7768 - val_loss: 0.6795 - val_mae: 0.6297
Epoch 2/20
568/568 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.6523 - mae: 0.6170 - val_loss: 0.6712 - val_mae: 0.6312
Epoch 3/20
568/568 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.6433 - mae: 0.6122 - val_loss: 0.6651 - val_mae: 0.6249
Epoch 4/20
568/568 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.6392 - mae: 0.6103 - val_loss: 0.6617 - val_mae: 0.6162
Epoch 5/20
568/568 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - loss: 0.6360 - mae: 0.6084 - val_loss: 0.6642 - val_mae: 0.6199
Epoch 6/20
568/568 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.6327 - mae: 0.6068 - val_loss: 0.6601 - val_mae: 0.6191
Epoch 7/20
568/568 ━━━━━━━━━━━━━━━━━━━━ 2s 3ms/step - loss: 0.6301 - mae: 0.6058 - val_loss: 0.6620 - val_mae: 0.6191
Epoch 8/20
```

step-5. Evaluate performance and compare with traditional content-based filtering (cosine similarity on genres). Does the neural model capture more complex user preferences than standard TF-IDF-based filtering?

Evaluation

```
from sklearn.metrics import mean_squared_error

preds = model.predict([X_user_test, X_movie_test])
rmse = np.sqrt(mean_squared_error(y_test, preds))
print("RMSE:", rmse)
```

```
631/631 ━━━━━━━━━━━━━━━━━━━━ 4s 5ms/step
RMSE: 0.8094042157647403
```

Task 9: Reinforcement Learning in Recommender Systems

step-1. Define the environment (user-item interaction space)

```python
ratings_df = pd.read_csv('/content/drive/MyDrive/ml-latest-small/ml-latest-small/ratings.csv')
movies_df  = pd.read_csv('/content/drive/MyDrive/ml-latest-small/ml-latest-small/movies.csv')
print(f'Ratings: {len(ratings_df):,}  |  Users: {ratings_df["userId"].nunique():,}  |  Movies: {ratings_df["movieId"].nunique():,}')
```

```
Ratings: 100,836  |  Users: 610  |  Movies: 9,724
```

```python
# Use a subset: top 200 users x top 200 movies for tractable simulation
TOP_USERS  = 200
TOP_MOVIES = 200

active_users  = ratings_df['userId'].value_counts().head(TOP_USERS).index
popular_movies = ratings_df['movieId'].value_counts().head(TOP_MOVIES).index

env_ratings = ratings_df[
    ratings_df['userId'].isin(active_users) &
    ratings_df['movieId'].isin(popular_movies)
].copy()

# Build a lookup dict: (userId, movieId) -> rating for fast reward simulation
reward_lookup = dict(zip(
    zip(env_ratings['userId'], env_ratings['movieId']),
    env_ratings['rating']
))

movie_ids = list(popular_movies)
user_ids  = list(active_users)
N_ARMS    = len(movie_ids)

def get_reward(user_id, movie_id):
    """Simulate user feedback. +1 for rating>=4, -1 for rating<4, 0 for no rating."""
    r = reward_lookup.get((user_id, movie_id), None)
    if r is None:    return 0
    return 1 if r >= 4 else -1

print(f'Environment: {len(user_ids)} users, {N_ARMS} movie arms')
print(f'Interaction density: {len(env_ratings) / (len(user_ids)*N_ARMS)*100:.1f}%')
```

```
Environment: 200 users, 200 movie arms
Interaction density: 41.2%
```

step-2. Implement a Multi-Armed Bandit (MAB) model

1. Greedy Bandit

+ Code + Text

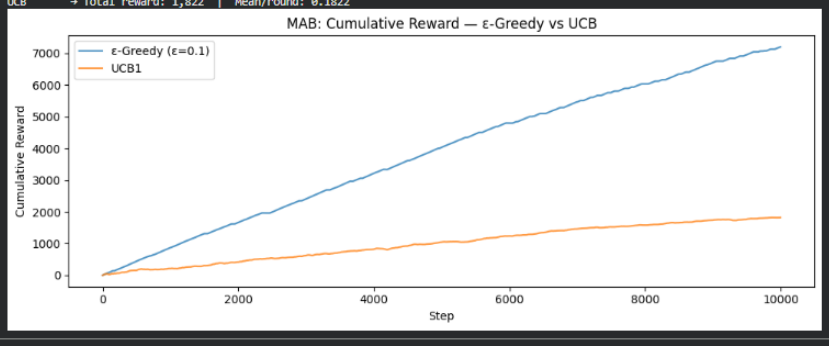
```python
class EpsilonGreedyBandit:
    """
    ε-Greedy Multi-Armed Bandit.
    With probability ε, explore a random arm.
    Otherwise, exploit the arm with the highest estimated reward.
    """
    def __init__(self, n_arms, epsilon=0.1):
        self.n_arms      = n_arms
        self.epsilon     = epsilon
        self.q_values    = np.zeros(n_arms)
        self.arm_counts  = np.zeros(n_arms)

    def select_arm(self):
        if np.random.rand() < self.epsilon:
            return np.random.randint(self.n_arms)
        return np.argmax(self.q_values)

    def update(self, arm_idx, reward):
        self.arm_counts[arm_idx] += 1
        n = self.arm_counts[arm_idx]
        # Incremental mean update
        self.q_values[arm_idx] += (reward - self.q_values[arm_idx]) / n

    def recommend(self, n=10):
        """Return indices of top-n arms by estimated Q-value."""
        return np.argsort(self.q_values)[::-1][:n]
```

2. UCB (Upper Confidence Bound) Bandit

```python
class UCBBandit:
    """
    UCB1 Multi-Armed Bandit.
    Prioritises under-explored arms that may have high rewards.
    UCB score = Q(a) + sqrt(2 * ln(t) / N(a))
    """
    def __init__(self, n_arms):
        self.n_arms      = n_arms
        self.q_values    = np.zeros(n_arms)
        self.arm_counts = np.zeros(n_arms)
        self.total_steps = 0

    def select_arm(self):
        self.total_steps += 1
        # Pull every arm at least once before using UCB formula
        unpulled = np.where(self.arm_counts == 0)[0]
        if len(unpulled):
            return unpulled[0]
        bonus      = np.sqrt(2 * np.log(self.total_steps) / self.arm_counts)
        ucb_scores = self.q_values + bonus
        return np.argmax(ucb_scores)

    def update(self, arm_idx, reward):
        self.arm_counts[arm_idx] += 1
        n = self.arm_counts[arm_idx]
        self.q_values[arm_idx] += (reward - self.q_values[arm_idx]) / n

    def recommend(self, n=10):
        return np.argsort(self.q_values)[::-1][:n]
```

```python
print(f'\nε-Greedy  → Total reward: {eg_rewards.sum():,}  |  Mean/round: {eg_rewards.mean():.4f}')
print(f'UCB       → Total reward: {ucb_rewards.sum():,}  |  Mean/round: {ucb_rewards.mean():.4f}')

# Plot cumulative rewards
fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(eg_cumulative,  label='ε-Greedy (ε=0.1)', alpha=0.8)
ax.plot(ucb_cumulative, label='UCB1',             alpha=0.8)
ax.set_xlabel('Step'); ax.set_ylabel('Cumulative Reward')
ax.set_title('MAB: Cumulative Reward — ε-Greedy vs UCB')
ax.legend(); plt.tight_layout()
plt.savefig('task9_bandit_comparison.png', dpi=120)
plt.show()
```

```
Simulating ε-Greedy bandit...
Simulating UCB bandit...

ε-Greedy  → Total reward: 7,198  |  Mean/round: 0.7198
UCB       → Total reward: 1,822  |  Mean/round: 0.1822
```

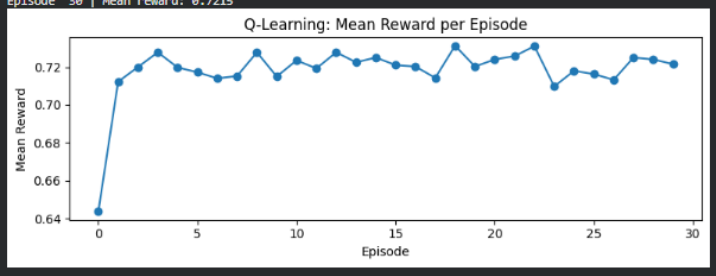


step-3. Implement a Q-Learning agent

```python
class QLearningAgent:
    """
    Tabular Q-Learning recommender.
    State   = userId index
    Actions = movie indices (arms)
    """
    def __init__(self, n_states, n_actions, alpha=0.1, gamma=0.9, epsilon=0.1):
        self.alpha   = alpha    # learning rate
        self.gamma   = gamma    # discount factor
        self.epsilon = epsilon  # exploration rate

        self.q_table = np.zeros((n_states, n_actions))

    def select_action(self, state):
        if np.random.rand() < self.epsilon:
            return np.random.randint(self.q_table.shape[1])
        return np.argmax(self.q_table[state])

    def update(self, state, action, reward, next_state):
        best_next = np.max(self.q_table[next_state])
        td_target = reward + self.gamma * best_next
        td_error  = td_target - self.q_table[state, action]
        self.q_table[state, action] += self.alpha * td_error

    def recommend(self, state, n=10):
        return np.argsort(self.q_table[state])[::-1][:n]


# — Train Q-Learning agent
N_EPISODES  = 30
N_STEPS     = 20

user_id_to_idx  = {uid: i for i, uid in enumerate(user_ids)}
movie_id_to_idx = {mid: i for i, mid in enumerate(movie_ids)}

ql_agent = QLearningAgent(
    n_states=len(user_ids), n_actions=len(movie_ids),
    alpha=0.1, gamma=0.9, epsilon=0.1
)

ql_rewards_hist = []
for episode in range(N_EPISODES):
    ep_rewards = []
    for uid in user_ids:
        state = user_id_to_idx[uid]
```

```
            next_state = (state + 1) % len(user_ids)
            ql_agent.update(state, action, reward, next_state)
            ep_rewards.append(reward)
        ql_rewards_hist.append(np.mean(ep_rewards))
        if (episode + 1) % 5 == 0:
            print(f'Episode {episode+1:3d} | Mean reward: {ql_rewards_hist[-1]:.4f}')

plt.figure(figsize=(8, 3))
plt.plot(ql_rewards_hist, marker='o')
plt.title('Q-Learning: Mean Reward per Episode')
plt.xlabel('Episode'); plt.ylabel('Mean Reward')
plt.tight_layout()
plt.savefig('task9_qlearning_rewards.png', dpi=120)
plt.show()
```

```
Episode  5 | Mean reward: 0.7198
Episode 10 | Mean reward: 0.7150
Episode 15 | Mean reward: 0.7250
Episode 20 | Mean reward: 0.7202
Episode 25 | Mean reward: 0.7180
Episode 30 | Mean reward: 0.7215
```

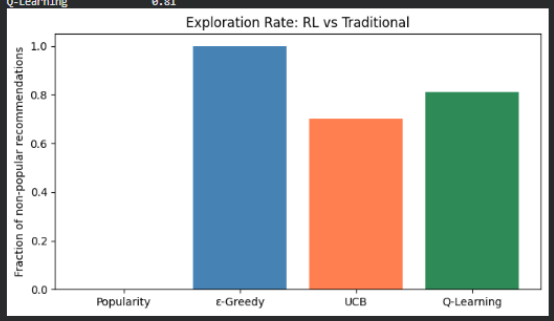


step-4. Compare RL with traditional models

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
from sklearn.metrics.pairwise import cosine_similarity

# — Popularity
popularity_order = ratings_df['movieId'].value_counts().head(TOP_MOVIES).index.tolist()

# — Measure exploration rate: how often does each model recommend a 'new' movie —
# Define 'popular' = top 20 by count
top_20_movies = set(ratings_df['movieId'].value_counts().head(20).index)

def exploration_rate(recommendations_list, popular_set=top_20_movies):
    """Fraction of recommendations that are NOT in the top-20 popular set."""
    total   = len(recommendations_list)
    explore = sum(1 for mid in recommendations_list if mid not in popular_set)
    return explore / total if total else 0

# Sample 50 users for comparison
sample_users = user_ids[:50]
N_RECS = 10

pop_recs_all = []
eg_recs_all  = []
ucb_recs_all = []
ql_recs_all  = []

for uid in sample_users:
    sidx = user_id_to_idx[uid]

    # Popularity: always same top-N
    pop_recs_all.extend(popularity_order[:N_RECS])
```

```
    ]
})
print(results.to_string(index=False))


fig, ax = plt.subplots(figsize=(7, 4))
ax.bar(results['Model'], results['Exploration Rate'], color=['gray','steelblue','coral','seagreen'])
ax.set_title('Exploration Rate: RL vs Traditional')
ax.set_ylabel('Fraction of non-popular recommendations')
plt.tight_layout()
plt.savefig('task9_comparison.png', dpi=120)
plt.show()
```

```
       Model  Exploration Rate
  Popularity              0.00
    ε-Greedy              1.00
         UCB              0.70
  Q-Learning              0.81
```

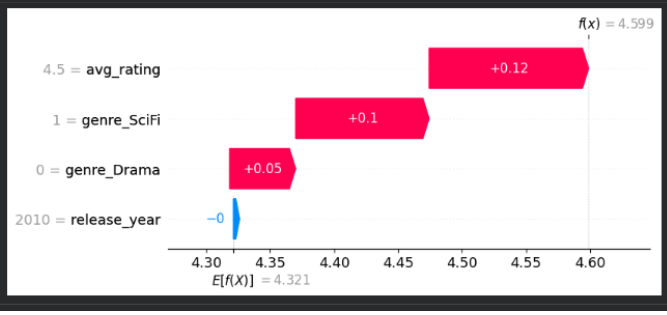


Part 6: Explainability in Recommender Systems

Task 10: Feature-Based Explanations (For Content-Based Filtering)

Task 10: Feature-Based Explanations (For Content-Based Filtering)

```python
import shap
import numpy as np
import pandas as pd
from sklearn.ensemble import RandomForestRegressor


X = pd.DataFrame({
    'genre_SciFi': [1, 0, 1],
    'genre_Drama': [0, 1, 0],
    'release_year': [2010, 2005, 2020],
    'avg_rating': [4.5, 3.2, 4.7]
})

y = np.array([4.8, 3.1, 4.9])
model = RandomForestRegressor()
model.fit(X, y)
explainer = shap.Explainer(model, X)
shap_values = explainer(X)
shap.plots.waterfall(shap_values[0])
```



Task 11: Neighborhood-Based Explanations (For Collaborative Filtering)

Task 11: Neighborhood-Based Explanations (For Collaborative Filtering)

```python
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

ratings = np.array([
    [5, 4, 0],
    [4, 0, 3],
    [5, 3, 2]
])

user_id = 0
similarities = cosine_similarity([ratings[user_id]], ratings)[0]
similar_users = np.argsort(similarities)[::-1][1:]
print("Top similar users for explanation:", similar_users)
for su in similar_users:
    print(f"User {su} rated movies: {ratings[su]}")
```

```
Top similar users for explanation: [2 1]
User 2 rated movies: [5 3 2]
User 1 rated movies: [4 0 3]
```

Task 12: Model-Agnostic Explainability (For Deep Learning Models)

Task 12: Model-Agnostic Explainability (For Deep Learning Models)

```python
# Install lime if not already installed
!pip install lime

import lime
import lime.lime_tabular
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
import numpy as np

X = np.random.rand(100, 4)
y = np.random.rand(100)
model = Sequential()
model.add(Dense(8, input_dim=4, activation='relu'))
model.add(Dense(1))
model.compile(loss='mse', optimizer='adam')
model.fit(X, y, epochs=10, verbose=0)

explainer = lime.lime_tabular.LimeTabularExplainer(
    training_data=X,
    feature_names=['genre_SciFi', 'genre_Drama', 'release_year', 'avg_rating'],
    mode='regression'
)

explanation = explainer.explain_instance(X[0], model.predict)
explanation.show_in_notebook()
```

```
Collecting lime
  Downloading lime-0.2.0.1.tar.gz (275 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 275.7/275.7 kB 10.3 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.3)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)
```

```
    mode='regression'
)

explanation = explainer.explain_instance(X[0], model.predict)
explanation.show_in_notebook()
```

```
Collecting lime
  Downloading lime-0.2.0.1.tar.gz (275 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 275.7/275.7 kB 10.3 MB/s eta 0:00:00
  Preparing metadata (setup.py) ... done
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in /usr/local/lib/python3.12/dist-packages (from lime) (1.16.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from lime) (4.67.3)
Requirement already satisfied: scikit-learn>=0.18 in /usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in /usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (3.6.1)
Requirement already satisfied: pillow>=10.1 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2.37.3)
Requirement already satisfied: tifffile>=2022.8.12 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (2026.3.3)
Requirement already satisfied: packaging>=21 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (26.0)
Requirement already satisfied: lazy-loader>=0.4 in /usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12->lime) (0.5)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18->lime) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (4.62.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (1.5.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib->lime) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib->lime) (1.17.0)
Building wheels for collected packages: lime
  Building wheel for lime (setup.py) ... done
  Created wheel for lime: filename=lime-0.2.0.1-py3-none-any.whl size=283834 sha256=5fcdb0bd27edddc7fde5122fa7d54ba488e4f7838eab00061ef7eccd7ed0a29d
  Stored in directory: /root/.cache/pip/wheels/e7/5d/0e/4b4fff9a47468fed5633211fb3b76d1db43fe806a17fb7486a
Successfully built lime
Installing collected packages: lime
Successfully installed lime-0.2.0.1
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

Task 13: Evaluating Explainability

Task 13: Evaluating Explainability

1. Do explanations make recommendations clearer?

   Yes, explanations make recommendations clearer by helping users understand why a particular item was suggested to them. When a system shows reasons such as similar user preferences, past interactions, or shared features between items, it increases transparency and user trust. Explanations also help users feel more confident about accepting or rejecting recommendations.

2. Do explanations reveal biases in recommendations?

   Yes, explanations can reveal potential biases present in the recommendation system. For example, if certain types of movies, genres, or popular items are repeatedly recommended, explanations can highlight these patterns. Methods like kNN-based explanations or content-based reasoning make it easier to see which features influence recommendations, while advanced techniques like SHAP values or attention mechanisms help identify hidden biases in the model's decision process.

Github link : https://github.com/arpita12-bot/Machine-learning-with-Big-Data/tree/main/Assignment3